MANOJ KUMAR

LCI2024026

ASSIGNMENT – 3 (APL)

CODE)

```
# cook your dish here
from collections import Counter

# 1. The Source String S (Containing the assignment text)
S = """
1. Introduction to Frequency Analysis Workflows
Frequency analysis serves as the foundational "Hello World" of natural language processing (NLP)
and data science because it represents the first step in converting unstructured, qualitative text into
structured, quantitative data. By calculating the occurrence of discrete tokens, we transform a raw
stream of characters into actionable insights that reveal patterns, themes, and statistical significance.

In high-performance Python engineering, this process is best executed through a five-stage pipeline:
Tokenization, Normalization, Filtering, Tracking, and Ordering. This workflow demonstrates a
sophisticated "synergy of collections," where data undergoes a metamorphosis across Python's
built-in types. We move from immutable Strings to mutable Lists, utilize Sets for optimized
membership testing, aggregate state within Dictionaries, and finally leverage Tuples for data
integrity during ranking. Selecting the correct structure for each specific computational task is the
hallmark of performant, idiomatic Python. The journey from raw data to insight begins with breaking down the continuous, immutable stream
of text into manageable, discrete units.

2. Stage 1: Tokenization (Strings and the Stream of Data)
Tokenization is the process of converting a singular, immutable string a sequence of characters-
into a sequence of discrete entities called tokens. Python strings are immutable sequences; they
cannot be changed in-place. Because we cannot "clean" or "modify" a string directly to remove
noise, we must first partition it into a mutable container to allow for individual analysis and
transformation. Using the "Splitting and joining" methods of the string type, we employ S.split() to partition
the data. By default, this method returns a list of words using any whitespace string as a delimiter.

Solved Example: Tokenizing Raw Text
#S: The raw string source
S = "Python is a multiparadigm language. Python is performant, Python is idiomatic!"
```

```
# Execute split to create a list of tokens

tokens = S.split()

print (tokens)


# Output: ['Python', 'is', 'a', 'multiparadigm', 'language.', 'Python', 'is', 'performant,', 'Python', 'is', 'idiomatic!']
```

The "So What?" Layer

The choice of delimiter determines the granularity of your data stream. While S.split() handles

general whitespace effectively, it leaves punctuation attached to the words (e.g., "language." vs

"language"). You must evaluate whether the default behavior meets your requirements or if

explicit delimiters or subsequent cleaning phases are necessary to prevent "performant," and

"performant" from being counted as distinct entities. Because strings are immutable, we have now "cast" our data into a mutable List, allowing us to

begin the normalization process.

3. Stage 2: Normalization (Lists and Comprehensions)

To ensure statistical accuracy, data must be made uniform. Stylistic variations like "Python" and

"python" must be treated as the same entity. Normalization involves transforming tokens into a

standard format, typically by lowercasing and stripping trailing punctuation. Python's List Comprehensions are the preferred tool here. List comprehensions perform an

implied loop and collect expression results in a new list. This is an optimized expression that is

significantly more performant than a standard for loop, which incurs the overhead of repeated

.append() calls.

Solved Example: Normalizing Tokens

```
#L: The token list from Stage 1

#Apply .lower() and strip() to handle case and basic punctuation

L= [t.lower().strip('.,!') for t in tokens]

print (L)


# Output: ['python', 'is', 'a', 'multiparadigm', 'language', 'python', 'is', 'performant', 'python', 'is', 'idiomatic']
```

The "So What?" Layer

By utilizing a list comprehension, we execute in-memory transformations in a single, highly

optimized step. However, even a "clean" list contains "noise"-high-frequency, low-meaning words

known as stop words. To remove these efficiently, we must transition to a structure optimized for

membership testing.

4. Stage 3: Filtering (Sets and Membership Testing)

Production-scale NLP requires computational efficiency. Checking a list of tokens against a list of "stop words" (like "is" or "a") is a performance trap. In a List, membership testing (x in L) requires an O(N) linear scan.

Instead, we use the Set data type. Sets are unordered collections of unique, immutable objects implemented as hash tables. This structural choice provides 0(1) average-time complexity for membership testing.

Solved Example: Filtering with Sets

```
# Define a set of stop words for 0(1) lookup
stop_words = {"is", "a", "the", "an"}
# Filter the normalized list L
filtered_tokens = [w for w in L if w not in stop_words]
print (filtered_tokens)

# Output: ['python', 'multiparadigm', 'language', 'python', 'performant', 'python', 'idiomatic']
```

The "So What?" Layer

Using a Set for filtering is not just a "best practice" it is a requirement for scalability. When your stop-word list grows to hundreds of entries and your token stream to millions, the O(l) lookup speed of a hash table prevents the pipeline from becoming a bottleneck. Once the data is refined, we must aggregate these occurrences into a stateful mapping.

4. Stage 4: Frequency Tracking (Dictionaries and Counters)

To associate unique keys (words) with mutable values (counts), we use a Dictionary. The Python dictionary is a "dynamically expandable hash table", making it the ideal mapping structure for stateful aggregation. While a standard dict could be used with D.get(k, 0) + 1, the standard library's collections. Counter is the professional choice for reducing boilerplate and minimizing state retention errors.

Solved Example: Aggregating Frequencies

```
from collections import Counter
# Input the filtered list into a Counter object
word_freq = Counter (filtered_tokens)
print (word_freq)

#Output: Counter({'python': 3, 'multiparadigm': 1, 'language': 1, 'performant': 1, 'idiomatic': 1})
```

The "So What?" Layer

Counter abstracts the "initialize-and-increment" logic. This prevents the common "KeyError" when accessing a word for the first time. Your goal is to use high-level abstractions like Counter to ensure code maintainability while relying on the underlying dictionary's hash table for efficient O(1) access and updates.

5. Stage 5: Ordering and Ranking (Tuples and Decorated Sorting)

The final stage of the pipeline is identifying the most significant data points. Because dictionaries are inherently unordered mappings, we must transform the data into a sequence of Tuples for sorting. We use "decorated sorting" via the key parameter to sort by frequency (the second item in the tuple) rather than alphabetically. For maximum memory efficiency in large systems, we use L.sort(), which modifies the list in-place, rather than the sorted () function, which creates an entirely new list.

Solved Example: In-Place Decorated Sorting

```
# Convert dictionary items to a list of (word, count) tuples
#T: A list of tuples for integrity
T = list(word_freq.items())

#Sort in-place by count (index 1), descending
T.sort(key=lambda x: x[1], reverse=True)
print (T)

# Output: [('python', 3), ('multiparadigm', 1), ('language', 1), ('performant', 1), ('idiomatic', 1)]
```

The "So What?" Layer

We utilize Tuples specifically for their integrity. As immutable sequences, they ensure that the association between a word and its count cannot be accidentally decoupled during the sorting operation. By using the key parameter with a Lambda expression, we "decorate" the sort, prioritizing the statistical weight (the frequency) over the lexicographical value of the token.

Conclusion: The Synthesis of Pythonic Collections

Robust data engineering in Python is achieved by matching algorithmic requirements to the specific functional strengths of built-in data structures. By understanding the transition from immutable streams to mutable lists, high-speed set lookups, and stateful dictionary mappings, you can build tools that are both performant and idiomatic."""

```
# --- The 5-Stage Pipeline ---
```

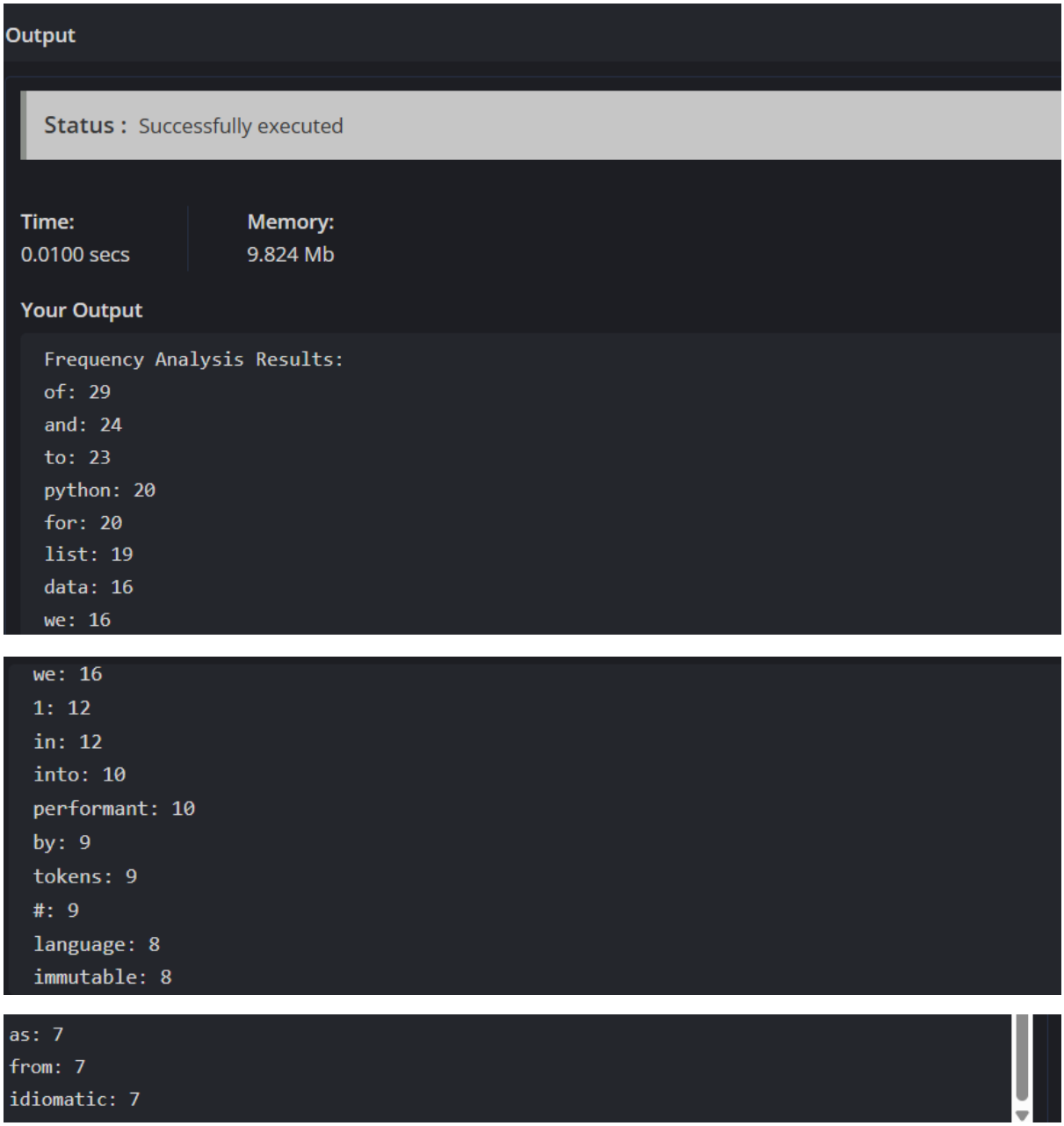
```
# Stage 1: Tokenization

tokens = S.split()


# Stage 2: Normalization (removing basic punctuation like in the assignment example)

L = [t.lower().strip('.,!()"\'[]{}') for t in tokens]


# Stage 3: Filtering (using the stop words set from the assignment)

stop_words = {"is", "a", "the", "an"}

filtered_tokens = [w for w in L if w not in stop_words]


# Stage 4: Frequency Tracking

word_freq = Counter(filtered_tokens)


# Stage 5: Ordering and Ranking

T = list(word_freq.items())

T.sort(key=lambda x: x[1], reverse=True)


# Output the results (Printing top 20 for readability)

print("Frequency Analysis Results:")

for word, count in T[:20]:

  # Ignoring empty strings that might result from stripping standalone punctuation

  if word:

    print(f"{word}: {count}")
```

Python3

```
# cook your dish here
from collections import Counter

# 1. The Source String S (Containing the assignment text)
S = """
1. Introduction to Frequency Analysis Workflows
Frequency analysis serves as the foundational "Hello World" of natural language processing (NLP)
and data science because it represents the first step in converting unstructured, qualitative text into
structured, quantitative data. By calculating the occurrence of discrete tokens, we transform a raw
stream of characters into actionable insights that reveal patterns, themes, and statistical significance.

In high-performance Python engineering, this process is best executed through a five-stage pipeline:
Tokenization, Normalization, Filtering, Tracking, and Ordering. This workflow demonstrates a
sophisticated "synergy of collections," where data undergoes a metamorphosis across Python's
built-in types. We move from immutable Strings to mutable Lists, utilize Sets for optimized
membership testing, aggregate state within Dictionaries, and finally leverage Tuples for data
integrity during ranking. Selecting the correct structure for each specific computational task is the
hallmark of performant, idiomatic Python. The journey from raw data to insight begins with breaking down the continuous, immutable stream
of text into manageable, discrete units.

2. Stage 1: Tokenization (Strings and the Stream of Data)
Tokenization is the process of converting a singular, immutable string a sequence of characters-
into a sequence of discrete entities called tokens. Python strings are immutable sequences; they
cannot be changed in-place. Because we cannot "clean" or "modify" a string directly to remove
noise, we must first partition it into a mutable container to allow for individual analysis and
transformation. Using the "Splitting and joining" methods of the string type, we employ S.split() to partition
the data. By default, this method returns a list of words using any whitespace string as a delimiter.

Solved Example: Tokenizing Raw Text
#S: The raw string source
S = "Python is a multiparadigm language. Python is performant, Python is idiomatic!"
# Execute split to create a list of tokens
tokens = S.split()
print (tokens)

# Output: ['Python', 'is', 'a', 'multiparadigm', 'language.', 'Python', 'is', 'performant,', 'Python', 'is', 'idiomatic!']
```

```
The "So What?" Layer
The choice of delimiter determines the granularity of your data stream. While S.split() handles
general whitespace effectively, it leaves punctuation attached to the words (e.g., "language." vs
"language"). You must evaluate whether the default behavior meets your requirements or if
explicit delimiters or subsequent cleaning phases are necessary to prevent "performant," and
"performant" from being counted as distinct entities. Because strings are immutable, we have now "cast" our data into a mutable List, allowing us to
begin the normalization process.

3. Stage 2: Normalization (Lists and Comprehensions)
To ensure statistical accuracy, data must be made uniform. Stylistic variations like "Python" and
"python" must be treated as the same entity. Normalization involves transforming tokens into a
standard format, typically by lowercasing and stripping trailing punctuation. Python's List Comprehensions are the preferred tool here. List
    comprehensions perform an
implied loop and collect expression results in a new list. This is an optimized expression that is
significantly more performant than a standard for loop, which incurs the overhead of repeated
.append() calls.

Solved Example: Normalizing Tokens
#L: The token list from Stage 1
#Apply .lower() and strip() to handle case and basic punctuation
L= [t.lower().strip('.,!') for t in tokens]
print (L)

# Output: ['python', 'is', 'a', 'multiparadigm', 'language', 'python', 'is', 'performant', 'python', 'is', 'idiomatic']

The "So What?" Layer
By utilizing a list comprehension, we execute in-memory transformations in a single, highly
optimized step. However, even a "clean" list contains "noise"-high-frequency, low-meaning words
known as stop words. To remove these efficiently, we must transition to a structure optimized for
membership testing.

4. Stage 3: Filtering (Sets and Membership Testing)
Production-scale NLP requires computational efficiency. Checking a list of tokens against a list of
"stop words" (like "is" or "a") is a performance trap. In a List, membership testing (x in L)
requires an O(N) linear scan.
Instead, we use the Set data type. Sets are unordered collections of unique, immutable objects
implemented as hash tables. This structural choice provides 0(1) average-time complexity for
membership testing.

Solved Example: Filtering with Sets
# Define a set of stop words for 0(1) lookup
stop_words = {"is", "a", "the", "an"}
# Filter the normalized list L
filtered_tokens = [w for w in L if w not in stop_words]
print (filtered_tokens)

# Output: ['python', 'multiparadigm', 'language', 'python', 'performant', 'python', 'idiomatic']

The "So What?" Layer
Using a Set for filtering is not just a "best practice" it is a requirement for scalability. When your
stop-word list grows to hundreds of entries and your token stream to millions, the O(l) lookup
speed of a hash table prevents the pipeline from becoming a bottleneck. Once the data is refined, we must aggregate these occurrences into a stateful
    mapping.

4. Stage 4: Frequency Tracking (Dictionaries and Counters)
To associate unique keys (words) with mutable values (counts), we use a Dictionary. The Python
dictionary is a "dynamically expandable hash table", making it the ideal mapping structure for
stateful aggregation. While a standard dict could be used with D.get(k, 0) + 1, the standard library's
collections. Counter is the professional choice for reducing boilerplate and minimizing state
retention errors.

Solved Example: Aggregating Frequencies
from collections import Counter
# Input the filtered list into a Counter object
word_freq = Counter (filtered_tokens)
print (word_freq)

#Output: Counter({'python': 3, 'multiparadigm': 1, 'language': 1, 'performant': 1, 'idiomatic': 1})

The "So What?" Layer
Counter abstracts the "initialize-and-increment" logic. This prevents the common "KeyError"
when accessing a word for the first time. Your goal is to use high-level abstractions like Counter
to ensure code maintainability while relying on the underlying dictionary's hash table for efficient
print (T)

# Output: [('python', 3), ('multiparadigm', 1), ('language', 1), ('performant', 1), ('idiomatic', 1)]

The "So What?" Layer
We utilize Tuples specifically for their integrity. As immutable sequences, they ensure that the
association between a word and its count cannot be accidentally decoupled during the sorting
operation. By using the key parameter with a Lambda expression, we "decorate" the sort,
prioritizing the statistical weight (the frequency) over the lexicographical value of the token.

Conclusion: The Synthesis of Pythonic Collections
Robust data engineering in Python is achieved by matching algorithmic requirements to the specific
functional strengths of built-in data structures. By understanding the transition from immutable
streams to mutable lists, high-speed set lookups, and stateful dictionary mappings, you can build
tools that are both performant and idiomatic."""

# --- The 5-Stage Pipeline ---

# Stage 1: Tokenization
tokens = S.split()

# Stage 2: Normalization (removing basic punctuation like in the assignment example)
L = [t.lower().strip('.,!()"\'[]{}') for t in tokens]

# Stage 3: Filtering (using the stop words set from the assignment)
stop_words = {"is", "a", "the", "an"}
filtered_tokens = [w for w in L if w not in stop_words]

# Stage 4: Frequency Tracking
word_freq = Counter(filtered_tokens)

# Stage 5: Ordering and Ranking
T = list(word_freq.items())
T.sort(key=lambda x: x[1], reverse=True)

# Output the results (Printing top 20 for readability)
print("Frequency Analysis Results:")
for word, count in T[:20]:
    # Ignoring empty strings that might result from stripping standalone punctuation
    if word:
        print(f"{word}: {count}")
```

OUTPUT)


Output
Status : Successfully executed
Time:
0.0100 secs
Memory:
9.824 Mb
Your Output
Frequency Analysis Results:
of: 29
and: 24
to: 23
python: 20
for: 20
list: 19
data: 16
we: 16
we: 16
1: 12
in: 12
into: 10
performant: 10
by: 9
tokens: 9
#: 9
language: 8
immutable: 8
as: 7
from: 7
idiomatic: 7